

Constructor

Indu Singh, AP, CSE, DTU

A member function with same name as its class is called constructor & it is used to initialize objects of that class type with a legal initial value.

Eg1. class Student
{
private:
int roll no;
float marks;

public:

Student()

{

roll no = 2;

marks = 87.6;

}

}

if this constructor is not defined, then we won't be able to access private data

members without

explicitly

specifying getData() funcⁿ.

int main()

{

Student

Senior = {

2

↑

87.6

↑

~~0~~, ~~0.0~~}

// ; illegal

since data

members not accessible.

}

* we have to call explicitly getData() funcⁿ to initialise private data members but through constructor, compiler will automatically initialize data member.

Constructor Having Arguments (Parameterized Constructor)

Eg:

```
class XYZ
```

```
{ int i;
```

```
public:
```

```
float j;
```

```
XYZ (int a, float b)
```

```
{ i = a;
```

```
j = b;
```

```
}
```

```
};
```

Test method

```
int main()
```

```
{
```

```
XYZ obj1 (2, 30.4);
```

Implicit method

```
XYZ obj2 = XYZ (2, 70.3);
```

Explicit method.

```
}
```

2nd method

* Parameterized constructor can also have default arguments.

Eg:

```
class A
```

```
{
```

```
int i;
```

```
float j;
```


public : $\rightarrow$ // prototype declaration with default arguments
 A (int a = 0 , float b = 200.0);
 };

A :: A (int a , float b)
 {
 i = a;
 j = b;
 }
 // constructor definition.

int main()

{ A obj1 (25 , 255.0);

A obj2 ; $\rightarrow$ // Takes default argument values.

DEFAULT CONSTRUCTOR

Eg:

Class Test

{ int a;

char b;

public:

Test (int i , char j)

{ a = i;

b = j;

};

This is not default constructor $\leftarrow$


```
int main()
```

```
{ Test Tarray [10];
```

Error bcz

default constructor not available.

```
Test obj (20, 'a'); ✓
```

ok

statement.

}

Solution

Default
constructor
code

```
public:
```

```
Test()
```

```
{ cout << "Default constructor  
being called";
```

```
}
```

```
Test (int i, char j)
```

```
{ a = i;
```

```
b = j;
```

```
cout << "Constructor with  
arguments called";
```

```
}
```

```
=};
```

```
int main()
```

```
{
```

```
Test Tarray [10]; ✓ ok now  
created;
```

```
Test obj (20, 'a');
```

```
}
```


CONSTRUCTOR OVERLOADING

Eg:

```
class Deposit
{
    long int principal;
    int time;
    float rate;
    float total-amt;
```

public:

Deposit();

Deposit(long P, int t, float r);

Deposit(long P, int t);

Deposit(long P, float r);

void calc-amt();

void display();

};

Deposit::Deposit()

{ principal = time = rate = 0.0; }

Deposit::Deposit(long P, int t, float r)

{ principal = P;

time = t;

rate = r;


```
Deposit :: Deposit (long P, int t)
```

```
{ principal = P;
```

```
  time = t;
```

```
  rate = 0.08;
```

```
}
```

```
Deposit :: Deposit (long P, float r)
```

```
{ principal = P;
```

```
  time = 2;
```

```
  rate = r;
```

```
}
```

```
void Deposit :: calc_amt (i)
```

```
{ total_amt = principal + (principal * time * rate) / 100;
```

```
}
```

```
void Deposit :: display ()
```

```
{ cout << "Principal Amount" << principal;
```

```
  cout << "Period of Investment" << time;
```

```
  cout << "Rate of Interest" << rate;
```

```
  cout << "Total Amount" << total_amt;
```

```
}
```



```
void main()
```

```
{ Deposit D1;
```

```
Deposit D2 (2000, 2, 0.02);
```

```
Deposit D3 (5000, 3);
```

```
Deposit D4 (3000, 0.04);
```

```
D1. calc_amt();
```

```
D2. calc_amt();
```

```
D3. calc_amt();
```

```
D4. calc_amt();
```

```
cout << endl;
```

```
D1. display();
```

```
D2. display();
```

```
D3. display();
```

```
D4. display();
```

```
}
```


Making Use of Default Arguments.

Eg.

```
void amount (float princ, int time = 2,  
             float rate = 0.05);  
  
void amount (float princ, int time, float rate)  
{  
    cout << "Principal amount" << princ;  
    cout << " time" << time;  
    cout << " Rate" << rate;  
    cout << " Interest Amount" << (princ * rate  
                                     * time)  
}  
  
void main()  
{  
    cout << "Case 1";  
    amount (2000);  
    cout << "Case 2";  
    amount (2500, 4);  
    cout << "Case 3";  
    amount (2300, 4, 0.12);  
    cout << "Case 4";  
    amount (2500, 0.08);  
}
```